

JOBSHEET WEEK 2 WEB PROGRAMMING



SIMPUS-Mini

CSS3 Styling Basics for a Mini Library System

**DEPARTMENT OF INFORMATION TECHNOLOGY
STATE POLYTECHNIC OF MALANG
2026**

Table of Contents

Introduction	4
What Is New in Jobsheet 2?	4
Folder Structure	4
How to Read This Document	4
1. Core CSS Concepts	5
1.1 What Is CSS?	5
1.2 Anatomy of One CSS Rule	5
1.3 Linking CSS to HTML	5
1.4 Types of Selector Used in style.css	6
1.5 Every HTML Element Is a Box: The Box Model	6
1.6 Units of Measurement Used	7
1.7 Colours in Hex Format	7
2. What Changes in the HTML Files?	8
2.1 The CSS Path Differs per File	8
2.2 Why Is the HTML Structure Deliberately Left Unchanged?	8
3. CSS: Reset & Basic Body Styles	9
3.1 CSS Reset with the Universal Selector *	9
3.2 Basic <body> Styles	9
3.3 Link Styles (<a>)	10
4. CSS: Header & Navbar with Flexbox	11
4.1 The CSS Code	11
4.2 What Is Flexbox?	11
4.3 Enabling Flexbox: display: flex	11
4.4 Positioning the Flex Items	11
4.5 Padding on the Header	12
4.6 Nested Flexbox: the Navbar Inside the Header	12
4.7 Why Does the Selector header nav a "Win" over a?	13
5. CSS: main & section Layout	14
5.1 The CSS Code	14
5.2 Limiting the Width & Centring the Content (main)	14
5.3 A White Card for Every <section>	14
5.4 Headings Inside a Section	15
6. CSS: Statistic Cards with CSS Grid	16
6.1 Recalling the HTML	16
6.2 The CSS Code	16
6.3 The Selector :nth-of-type(2), Selecting "Only the Second Section"	16
6.4 Enabling Grid: display: grid	17
6.5 Flexbox vs Grid, What Is the Difference?	17
6.6 Styling the Contents of Each Card (<article>)	17
6.7 Why Not Just Use class?	18
7. CSS: Styling the Table	19
7.1 The CSS Code	19
7.2 Table Width & border-collapse	19
7.3 Header Cells (th) and Data Cells (td)	20
7.4 A Coloured Table Header	20
7.5 Alternating Rows & the Hover Effect	20
7.6 Action Buttons (Edit & Delete)	21

8. CSS: Styling the Form	22
8.1 The CSS Code	22
8.2 Spacing Between Fields	22
8.3 The Label as Its Own Block	22
8.4 Input Boxes & Dropdowns	23
8.5 The Submit Button	23
9. CSS: Footer	25
9.1 The CSS Code	25
9.2 Explanation	25
10. Summary & Further Exercises	26
10.1 Overall Summary of Jobsheet 2	26
10.2 Core Concepts to Remember	26
10.3 How to Try It Yourself	26
10.4 Exercises	27

Introduction

This documentation continues the Jobsheet 1 documentation and is meant for students who are new to HTML/CSS. If you are not yet comfortable with the basic HTML structure (`header`, `nav`, `main`, `table`, `form`, and so on), read the Jobsheet 1 documentation first before continuing here.

What Is New in Jobsheet 2?

As stated in the README of this jobsheet, **the HTML structure does not change at all** from jobsheet-01, only one new file is added, `assets/css/style.css`, plus one line of `<link rel="stylesheet">` on every HTML page. So the whole of this jobsheet 2 documentation focuses on **CSS**: what CSS is, how it works, and a line-by-line explanation of the `style.css`.

Folder Structure

```
jobsheet-02/
├── index.html
├── assets/
│   └── css/
│       └── style.css      # New file in this jobsheet
├── books/
│   ├── list.html
│   └── add.html
├── members/
│   ├── list.html
│   └── add.html
├── README.md
└── docs/                  # This documentation folder
```

1. Core CSS Concepts

Before dissecting `style.css`, get to know the CSS terms that will keep appearing throughout this documentation.

1.1 What Is CSS?

CSS (Cascading Style Sheets) is a language for controlling the **appearance** of HTML elements — colour, size, spacing, layout, and so on. While HTML determines the **structure/content** of the page (headings, paragraphs, tables, forms), CSS determines **what it looks like** on screen.

1.2 Anatomy of One CSS Rule

```
header {  
  background-color: #1d5b8a;  
  color: #fff;  
}
```

Part	Example	Function
Selector	<code>header</code>	Determines which HTML element is given this style. Here it means "every <code><header></code> " tag.
Declaration	<code>background-color: #1d5b8a;</code>	One style rule, made up of a property and a value , ended with a semicolon <code>;</code> .
Property	<code>background-color</code>	The visual aspect being set (background colour, font size, and so on).
Value	<code>#1d5b8a</code>	The value given to that property.
Declaration block	<code>{ ... }</code>	The curly braces that wrap one or more declarations for the same selector.

1.3 Linking CSS to HTML

There are 3 ways to link CSS to HTML, but this jobsheet uses the **most recommended** one: a separate CSS file, linked through the `<link>` tag inside `<head>`:

```
<link rel="stylesheet" href="assets/css/style.css">
```

- `rel="stylesheet"` — tells the browser that the linked file is a style sheet.
- `href="..."` — the location of the CSS file. Just like the `<a href="...">` link in the jobsheet-01 documentation, this path is **relative** to the location of the HTML file — see the full explanation in chapter 2.

The advantage of separating CSS into its own file (compared with writing styles directly on every HTML tag): **one style.css file is shared by 5 HTML pages at once** — if you want to change the button colour, just change one line in `style.css`, and it changes on every page automatically.

1.4 Types of Selector Used in style.css

Selector Type	Example in style.css	Meaning
Tag/Element	<code>body, header, table</code>	Selects every element with that tag.
Universal	<code>*</code>	Selects every element without exception.
Descendant	<code>header nav ul, td button</code>	Selects the <code>ul</code> element that sits inside <code>nav</code> which is inside <code>header</code> . The word order = the nesting order in the HTML.
Pseudo-class	<code>a:hover, tbody tr:hover</code>	Selects an element in a particular condition/state — <code>:hover</code> means "while the mouse cursor is over this element".
Positional pseudo-class	<code>tbody tr:nth-child(even), main section:nth-of-type(2)</code>	Selects elements by their order/position among their siblings. Discussed in detail in chapters 6 and 7.
Attribute	<code>form button[type="submit"]</code>	Selects the <code>button</code> element whose attribute type has the value <code>"submit"</code> .

1.5 Every HTML Element Is a Box: The Box Model

Every HTML element in the browser is really a **box**, made up of 4 layers from the inside out:

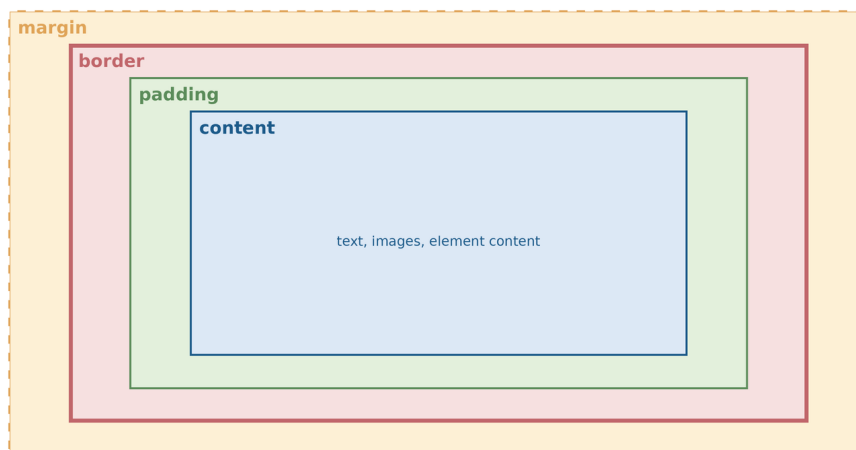


Figure 1.1 — The box model: margin, border, padding, and content.

- **margin** — the empty space **outside** the box, towards neighbouring elements.
- **border** — the line at the edge of the box.
- **padding** — the empty space **inside** the box, between the border and the content.
- **content** — the actual content (text, images, and so on).

The `box-sizing: border-box;` property (used on the first line of `style.css`) changes how the width/height of an element is calculated. This is an important concept explained in detail in the next chapter.

1.6 Units of Measurement Used

In `style.css` you will find several different units of measurement:

Unit	Example	Meaning
px (pixel)	<code>border-radius: 8px;</code>	A fixed/absolute size that does not change with anything else.
rem	<code>padding: 1rem 1.5rem;</code>	A size that is relative to the root font size (<code><html></code>), usually <code>1rem</code> = 16px by default. More flexible, because it adapts when the user changes the browser font size.
%	<code>width: 100%;</code>	A size that is relative to its parent element (for example 100% of the width of its wrapper box).
fr (fraction)	<code>grid-template-columns: repeat(3, 1fr);</code>	A unit specific to CSS Grid: "one fractional part" of the available space. Discussed in chapter 6.

1.7 Colours in Hex Format

Colours in `style.css` are mostly written in **hex** (hexadecimal) format, for example `#1d5b8a` or `#fff`:

- The `#RRGGBB` format, the first two digits for red (Red), the next two for green (Green), the last two for blue (Blue). For example `#1d5b8a` is dark blue (the blue component is the most dominant).
- `#fff` is the short form of `#ffffff` (pure white), if all three pairs of digits are the same you may write only 3 digits.

2. What Changes in the HTML Files?

The only change in every HTML file is **one new line** inside `<head>`:

```
<link rel="stylesheet" href="assets/css/style.css">
```

2.1 The CSS Path Differs per File

Because the `style.css` file is kept in one location (`jobsheet-02/assets/css/style.css`), while the HTML files are spread across several folders at different depths, the `href` value on the `<link>` tag has to be adjusted:

HTML File	File Location	href Used
<code>index.html</code>	root folder (<code>jobsheet-02/</code>)	<code>assets/css/style.css</code>
<code>books/list.html</code>	inside the <code>books/</code>	<code>../assets/css/style.css</code>
<code>books/add.html</code>	inside the <code>books/</code>	<code>../assets/css/style.css</code>
<code>members/list.html</code>	inside the <code>members/</code>	<code>../assets/css/style.css</code>
<code>members/add.html</code>	inside the <code>members/</code>	<code>../assets/css/style.css</code>

The pattern is the same as the paths on `<a href="...">` in the navigation menu (see the jobsheet-01 explanation §1.5): `../` means "go up one folder" before going into `assets/css/style.css`.

The most common mistake beginners make when adding CSS to many pages in different folders is **forgetting to adjust the number of `../`**, and as a result the CSS file "does not load" and the page still looks plain, with no clear error message on screen (the error usually only shows up in the *Console/Network* tab of the browser DevTools).

2.2 Why Is the HTML Structure Deliberately Left Unchanged?

This is an important design decision to understand: CSS **should** be able to change a page's appearance completely **without** changing its HTML structure. This principle is called **separation of concerns** (separation of responsibilities), HTML takes care of the structure/meaning of the content, CSS takes care of the appearance. This jobsheet is deliberately designed that way so you can compare directly: the "Segoe UI" font, the blue colour on the header, the navbar aligned horizontally, the statistic cards neatly arranged in 3 columns, **all of that is purely the work of CSS**, not new HTML.

Try comparing a screenshot of `index.html` from jobsheet-01 (without CSS, the browser's default appearance, every element stacked plainly and vertically) with `index.html` from jobsheet-02 (with CSS) to see for yourself how large an effect CSS has on the appearance, even though the HTML is identical.

3. CSS: Reset & Basic Body Styles

This is the **very top** part of the `style.css` file — the foundation that affects the whole page.

3.1 CSS Reset with the Universal Selector *

```
/* ===== Reset & Base ===== */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

- `*` is the **universal selector** — meaning this rule applies to **every HTML element** without exception (`div`, `p`, `h1`, `button`, and so on).
- `margin: 0; padding: 0;` removes the default spacing the browser automatically gives to many elements (for example `<h1>`, `<p>`, `<ul>` have different default margins in each browser). This is called a **CSS reset**, "levelling the starting line" so the appearance is consistent across all browsers, before we set it again the way we want in the rules that follow.
- `box-sizing: border-box;`, this is the most important one to understand: by default (`content-box`), if you set `width: 200px` and then add `padding: 20px`, the **total** width of the box shown on screen becomes 240px (200 + 20 + 20 left and right). With `border-box`, `width: 200px` means the **total** width of the box (including padding & border) stays 200px, the padding is "eaten" from the inside. This is why almost every modern CSS developer always adds the `box-sizing: border-box;` rule at the start of their stylesheet.

The comment `/* ===== Reset & Base ===== */` above it is a **CSS comment** (wrapped in `/* ... */`), which does not affect the appearance at all — it is only a note for the programmer so the file is easy to navigate (compare with the HTML comment `<!-- ... -->`).

3.2 Basic `<body>` Styles

```
body {
  font-family: "Segoe UI", Arial, sans-serif;
  color: #2b2b2b;
  background-color: #f5f6f8;
  line-height: 1.5;
}
```

Property	Value	Explanation
<code>font-family</code>	<code>"Segoe UI", Arial, sans-serif</code>	A list of fonts in order of priority : the browser tries "Segoe UI" first; if that font is not available on the user's computer, it tries "Arial"; if that one is missing too, it uses whatever generic <code>sans-serif</code> font is available on the system. Always include a generic font at the end of the list as a guaranteed backup (<i>fallback</i>).
<code>color</code>	<code>#2b2b2b</code>	The text colour, a very dark grey (almost black, but not pure black <code>#000</code> , so it is softer on the eyes).

Property	Value	Explanation
<code>background-color</code>	<code>#f5f6f8</code>	The background colour of the whole page, a very light grey.
<code>line-height</code>	<code>1.5</code>	The spacing between lines of text, 1.5 times the font height. Without a unit it means a multiple of the element's own font size. This value makes text more comfortable to read than the browser default (usually around <code>1.2</code>).

Because this rule is attached to the `body` selector, and almost every other element sits **inside** `<body>`, the `font-family`, `color`, dan `line-height` properties are automatically "inherited" (*inherited*) by child elements such as `<p>`, `<h2>`, `<td>`, and so on, unless that element is given another CSS rule that overrides it specifically (for example `header h1` has its own `font-size`, explained in chapter 4).

3.3 Link Styles (`<a>`)

```
a {
  color: #1d5b8a;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}
```

- `a { color: #1d5b8a; }`, every link is given a blue colour (matching the header colour, see chapter 4), replacing the browser's default blue.
- `text-decoration: none;`, removes the underline that normally comes with a link, so the appearance is cleaner.
- `a:hover { text-decoration: underline; }`, ini **pseudo-class** (see core concepts §1.4): the underline only appears **while the mouse cursor is over it (hover)** the link. This gives the user visual feedback that the text can be clicked, without making the page look messy when the cursor is elsewhere.

4. CSS: Header & Navbar with Flexbox

This section is the first introduction to **Flexbox**, one of the most important *layout* systems in modern CSS.

4.1 The CSS Code

```
/* ===== Header & Navbar (Flexbox) ===== */
header {
  background-color: #1d5b8a;
  color: #fff;
  padding: 1rem 1.5rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
}

header h1 {
  font-size: 1.4rem;
}

header nav ul {
  list-style: none;
  display: flex;
  gap: 1.25rem;
}

header nav a {
  color: #fff;
  font-weight: 500;
}
```

4.2 What Is Flexbox?

By default, HTML elements such as `<h1>` and `<nav>` inside `<header>` are stacked **vertically** (one below the other), because both are *block* elements. What we actually want is: the "SIMPUS-Mini" title on the left, the navigation menu on the right, **side by side on one horizontal line**. This is where Flexbox comes in, a one-dimensional layout system that controls how child elements are arranged inside a wrapper box.

4.3 Enabling Flexbox: display: flex

```
header {
  display: flex;
  ...
}
```

Writing `display: flex;` on `<header>` turns `<header>` into a **flex container**, and **all of its direct children** (here: `<h1>` and `<nav>`) automatically become **flex items** arranged side by side horizontally (the default), no longer stacked vertically.

4.4 Positioning the Flex Items

Property	Value	Effect
<code>align-items: center;</code>	—	Aligns the flex items vertically in the centre of the cross axis (<i>cross axis</i>) — so that <code><h1></code> and <code><nav></code> are both vertically centred even when their heights differ.

Property	Value	Effect
<code>justify-content: space-between;</code>	—	Controls the spacing between flex items on the main axis (<i>main axis</i> , horizontal). space-between pushes the first item to the far left , the last item to the far right , and shares the remaining empty space between them. This is what puts <code><h1></code> on the left and <code><nav></code> on the right of the header.
<code>flex-wrap: wrap;</code>	—	Allows a flex item to move to a new line when there is not enough room (for example on a narrow phone screen), instead of being forced onto one line until it is cut off or squeezed too far.

4.5 Padding on the Header

```
padding: 1rem 1.5rem;
```

CSS allows you to write 2 values at once on the `padding` (or `margin`) property: the first value is **for top and bottom**, the second is **for left and right**. So `1rem 1.5rem` means padding of `1rem` on the top & bottom, and `1.5rem` on the left & right. This is a short way of writing 4 different values without having to write `padding-top`, `padding-right`, `padding-bottom`, `padding-left` one by one.

4.6 Nested Flexbox: the Navbar Inside the Header

Interestingly, Flexbox is used **twice, nested** here — once on `<header>` (arranging `h1` & `nav` horizontally), and once more inside `<nav>` `<ul>` to arrange the **individual menu items**:

```
header nav ul {
  list-style: none;
  display: flex;
  gap: 1.25rem;
}
```

- `header nav ul`, a descendant selector (see core concepts §1.4): it selects the `<ul>` element inside the `<nav>` that is inside the `<header>`.
- `list-style: none;`, removes the bullet points that come with a `<ul>` list by default, because in a navbar we do not need a bullet list.
- `display: flex;`, turns the `<ul>` (which contains the `<li>` menu items) into a flex container too, so all the `<li>` sit side by side horizontally instead of stacked vertically like an ordinary list.
- `gap: 1.25rem;`, a modern Flexbox/Grid property for giving a **uniform gap** between flex items, without having to set a `margin` manually on each item.

```
header nav a {
  color: #fff;
  font-weight: 500;
}
```

This overrides (*override*) the default link colour (`a { color: #1d5b8a; }` from chapter 3) **only for links inside the navbar**, so the menu text is white (`#fff`), contrasting with the dark blue header background. `font-weight: 500` makes the text slightly bolder than normal (`400`) but not as bold as `bold` (`700`).

4.7 Why Does the Selector `header nav a` "Win" over `a`?

This is a simple example of the concept of **CSS specificity** (*specificity*): a more specific/detailed selector (`header nav a`, naming 3 nested tags) beats a more general selector (`a`, only 1 tag), **regardless of the order they are written in the file**. So even though the rule `a { color: #1d5b8a; }` is written first in the file, the rule `header nav a` is still the one that decides the link colour inside the navbar, because it is more specific.

5. CSS: main & section Layout

This section sets the width of the main page content and turns every `<section>` into a white "card" that is lifted off the grey background.

5.1 The CSS Code

```
/* ===== Main Layout ===== */
main {
  max-width: 1000px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}

section {
  background-color: #fff;
  border-radius: 8px;
  padding: 1.5rem;
  margin-bottom: 1.5rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}

section h2 {
  margin-bottom: 1rem;
  color: #1d5b8a;
}
```

5.2 Limiting the Width & Centring the Content (main)

```
main {
  max-width: 1000px;
  margin: 2rem auto;
  padding: 0 1.5rem;
}
```

- `max-width: 1000px;`, the width of `<main>` **will never exceed** 1000px, even when the monitor is far wider. This is common practice so that text/content does not stretch unnaturally wide and become hard to read on a large screen (imagine reading a paragraph as wide as a 27-inch monitor, your eyes get tired quickly).
- `margin: 2rem auto;`, remember from chapter 4 that 2 values mean top-bottom then left-right. Here: a top and bottom margin of `2rem`, and a left and right margin of `auto`. The value `auto` on the left and right margin is the **classic CSS trick for centring an element horizontally**, the browser automatically shares the remaining empty space equally on the left and right of `<main>` (whose width is limited by the `max-width: 1000px` above), so the content always sits in the middle of the screen.
- `padding: 0 1.5rem;`, padding of `0` on the top and bottom, and `1.5rem` on the left and right. This leaves a little space from the edge of the screen so the content is not pressed against the side, especially on narrow screens (phone/tablet) where `max-width: 1000px` does not apply yet (the screen really is narrower than that).

5.3 A White Card for Every `<section>`

```
section {
  background-color: #fff;
  border-radius: 8px;
  padding: 1.5rem;
  margin-bottom: 1.5rem;
}
```

```
box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}
```

Property	Effect
<code>background-color: #fff;</code>	A white background, contrasting with the grey page background (<code>#f5f6f8</code> from <code>body</code> , chapter 3) — this is what makes each <code><section></code> look like a separate "card".
<code>border-radius: 8px;</code>	Makes the corners of the box curved/rounded by 8px instead of sharp right angles. A common visual effect for a modern, soft look.
<code>padding: 1.5rem;</code>	A single value means the same spacing on all four sides (top, bottom, left, right) — giving breathing room between the edge of the card and its contents.
<code>margin-bottom: 1.5rem;</code>	The gap below each card, separating it from the next card/element below it.
<code>box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);</code>	Gives a soft shadow around the card, making it look slightly "lifted" off the background. The values are: <code>0</code> (the shadow is not shifted left or right), <code>1px</code> (shifted 1px downwards), <code>3px</code> (blurred by 3px), and <code>rgba(0, 0, 0, 0.08)</code> (black with an opacity/transparency of only 8% — a very faint, soft shadow, not a harsh one).

A note about `rgba()`: this is an alternative colour format to hex (`#1d5b8a`) explained in core concepts §1.7. `rgba(R, G, B, A)` writes a colour as Red-Green-Blue numbers (0–255) plus an **Alpha** value (the level of transparency, from `0` = fully see-through to `1` = fully solid). This format is used here **because transparency is needed** for the shadow, something plain hex cannot do.

5.4 Headings Inside a Section

```
section h2 {
  margin-bottom: 1rem;
  color: #1d5b8a;
}
```

Every `<h2>` that sits inside a `<section>` (every section heading across the whole site, such as "Summary", "Book List", "Add Member") is given a gap of `1rem` below it (separating the heading from the content underneath) and a blue colour matching the header (`#1d5b8a`), giving a consistent "theme colour" throughout the application.

6. CSS: Statistic Cards with CSS Grid

This section introduces **CSS Grid**, another layout system (alongside Flexbox) used specifically to arrange the 3 statistic cards on the Home page (Total Books, Total Members, On Loan) into 3 columns side by side.

6.1 Recalling the HTML

From the jobsheet-01 HTML documentation, `index.html` has a `<main>` structure containing 2 `<section>`:

```
<main>
  <section>                                <!-- section 1: welcome -->
    <h2>Welcome...</h2>
    <p>...</p>
  </section>

  <section>                                <!-- section 2: statistics summary -->
    <h2>Summary</h2>
    <article>...Total Books...</article>
    <article>...Total Members...</article>
    <article>...On Loan...</article>
  </section>
</main>
```

Section **two** is the one holding the 3 `<article>` statistic cards, and it is the target of CSS Grid.

6.2 The CSS Code

```
/* ===== Statistic Cards (CSS Grid) ===== */
main section:nth-of-type(2) {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

main section:nth-of-type(2) article {
  background-color: #eef4fa;
  border-radius: 8px;
  padding: 1.25rem;
  text-align: center;
}

main section:nth-of-type(2) article h3 {
  font-size: 0.95rem;
  color: #55677a;
  margin-bottom: 0.5rem;
}

main section:nth-of-type(2) article p {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1d5b8a;
}
```

6.3 The Selector `:nth-of-type(2)`, Selecting "Only the Second Section"

```
main section:nth-of-type(2) { ... }
```

- Notice that the HTML of `index.html` has 2 `<section>` elements of the same tag type, without any `class` or `id` to tell them apart.

- `:nth-of-type(2)` is a pseudo-class that selects an element by **the order in which it appears** among siblings with **the same tag**. So `section:nth-of-type(2)` means "the section that comes 2nd among all the `<section>` elements of that type".
- Combined with `main` in front of it (a descendant selector), `main section:nth-of-type(2)` therefore means "the second section inside `<main>`", that is the "Summary" section (not the "Welcome" section, which comes first).

This is what the note in the README of this jobsheet refers to: *"The statistic cards on the Home page use `main section:nth-of-type(2)` as a 3-column grid"*.

6.4 Enabling Grid: `display: grid`

```
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 1rem;
```

- `display: grid;`, turns this section into a **grid container**. All of its direct children (here: the 3 `<article>` elements) automatically become **grid items** arranged according to the columns/rows that have been defined.
- `grid-template-columns: repeat(3, 1fr);`, defines **3 columns**, each with a width of `1fr`. `repeat(3, 1fr)` is shorthand for writing `1fr 1fr 1fr` three times. Because all three columns are the same `1fr`, the available space is divided **equally** between those 3 columns.
- `gap: 1rem;`, a uniform gap between the columns (and between rows if there is more than one row), just like the `gap` in the Flexbox navbar (chapter 4).

6.5 Flexbox vs Grid, What Is the Difference?

A fair question from a beginner: why does the navbar use Flexbox while the statistic cards use Grid, when both are "arranging elements side by side"?

	Flexbox	CSS Grid
Dimensions	1 dimension (rows or columns)	2 dimensions (rows and columns at the same time)
Suited to	Arranging a number of items that is not fixed/flexible in a single row/column (such as a navbar menu whose number of items can change)	Arranging a grid/box layout with column sizes decided precisely from the start (such as statistic cards that are always 3 columns side by side)

In this jobsheet the navbar uses Flexbox because it only needs to arrange menu items horizontally (1 direction), while the statistic cards use Grid because they need explicit control over "3 columns of equal width", the most common case where Grid is more concise than Flexbox.

6.6 Styling the Contents of Each Card (`<article>`)

```
main section:nth-of-type(2) article {
  background-color: #eef4fa;
  border-radius: 8px;
  padding: 1.25rem;
  text-align: center;
}
```

Every `<article>` (an individual card) is given a very light blue background (`#eef4fa`, different from the white background of the section itself so the card reads as a "box inside a box"), rounded corners, padding, and centred text (`text-align: center`).

```
main section:nth-of-type(2) article h3 {
  font-size: 0.95rem;
  color: #55677a;
  margin-bottom: 0.5rem;
}

main section:nth-of-type(2) article p {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1d5b8a;
}
```

- `<h3>` (the small heading, for example "Total Books") is made **smaller** (`0.95rem`, smaller than the normal text size of `1rem`) and coloured a blue-grey (`#55677a`), because this is only a label, not the main focus.
- `<p>` (the number, for example "12") is made much **larger** (`1.8rem`), bold (`font-weight: 700` = bold), and coloured the theme blue (`#1d5b8a`), because this is the main piece of information to highlight for the user. This difference in size and font weight between the small label and the large number is a common UI design technique called **visual hierarchy** (*visual hierarchy*), guiding the user's eye to the most important information first.

6.7 Why Not Just Use class?

As the README of this jobsheet notes: *"The CSS classes are generic (based on semantic tags + `nth-child`) so they can be reused on the Members pages without duplicating classes."* In other words, instead of adding `class="stat-grid"` by hand in the HTML and then writing `.stat-grid { display: grid; ... }` in the CSS, this jobsheet deliberately chooses a selector approach based on **structure & position** (`main section:nth-of-type(2)`). The advantage: the HTML stays clean without any extra `class` attributes, and the same styling automatically applies on other pages as long as the structure is similar. The drawback (which you need to be aware of as a learner): if the order of the `<section>` elements in the HTML changes (for example if the "Summary" section is moved to be the first section), this Grid styling **ends up on the wrong target** because `:nth-of-type(2)` is tied to position, not meaning. This is a trade-off worth understanding as you start making your own CSS design decisions later on.

7. CSS: Styling the Table

This section beautifies the `<table>` table in `books/list.html` dan `members/list.html`.

7.1 The CSS Code

```
/* ===== Table ===== */
table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  text-align: left;
  padding: 0.65rem 0.75rem;
  border-bottom: 1px solid #e2e6ea;
}

thead {
  background-color: #1d5b8a;
  color: #fff;
}

tbody tr:nth-child(even) {
  background-color: #f7f9fb;
}

tbody tr:hover {
  background-color: #eef4fa;
}

td button {
  padding: 0.35rem 0.7rem;
  margin-right: 0.35rem;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
}

td button:first-of-type {
  background-color: #f0ad4e;
  color: #fff;
}

td button:last-of-type {
  background-color: #d9534f;
  color: #fff;
}
```

7.2 Table Width & border-collapse

```
table {
  width: 100%;
  border-collapse: collapse;
}
```

- **width: 100%;** the table widens to fill the whole width of its wrapper box (the `<section>` card, see chapter 5), rather than only being as wide as its longest content, which is the default table behaviour.
- **border-collapse: collapse;** sets the border lines (`border`) between table cells to be **merged into a single thin line**, instead of appearing as two separate lines with a gap between them (the default behaviour of

`border-collapse: separate`). This is what makes the row dividers in the table look neat and thin instead of doubled.

7.3 Header Cells (th) and Data Cells (td)

```
th, td {
  text-align: left;
  padding: 0.65rem 0.75rem;
  border-bottom: 1px solid #e2e6ea;
}
```

- The selector `th, td` (separated by a comma) means the same rule is applied to **both** kinds of cell at once — a shorthand for when two different selectors need identical styling, rather than writing two separate blocks.
- `text-align: left;` text aligned left. This is written explicitly because `<th>` is centre-aligned by default (`center`) in most browsers, so this rule makes the alignment of `th` match `td` and keeps the heading and data columns neatly lined up.
- `border-bottom: 1px solid #e2e6ea;` a thin light grey line **only along the bottom** of each cell, separating one row from the row below (similar to the lines in a simple Excel table, without vertical lines between columns).

7.4 A Coloured Table Header

```
thead {
  background-color: #1d5b8a;
  color: #fff;
}
```

The column heading row (`<thead>`, see the jobsheet-01 HTML documentation) is given the same theme blue background as the page header, with white text, visually stating that this row holds the "column headings", different from the data rows below it.

7.5 Alternating Rows & the Hover Effect

```
tbody tr:nth-child(even) {
  background-color: #f7f9fb;
}

tbody tr:hover {
  background-color: #eef4fa;
}
```

- `tbody tr:nth-child(even)` is similar to `:nth-of-type(2)` in chapter 6, but `:nth-child(even)` selects **every even-numbered row** (rows 2, 4, 6, and so on) inside `<tbody>`, then gives them a very light grey background (`#f7f9fb`) slightly different from the white section background. This effect is called **zebra stripes** — a classic table design technique that helps the reader's eye **follow a single row of data horizontally** without "wandering off" to another row, especially in tables with many columns.
- `tbody tr:hover` the `:hover` pseudo-class, the same idea as `a:hover` in chapter 3: when the mouse cursor is moved over a row, that row's background turns a very light blue (`#eef4fa`, the same colour as the statistic card background in chapter 6), showing which row is currently "highlighted", which is useful especially when the table has many rows.

An important note: the `:hover` rule is placed **after** the `:nth-child(even)` rule in the CSS file. Because both have the same specificity (each is one pseudo-class plus `tr`), the order they are written in the file **decides** which rule wins when the cursor is over an even row, and because `:hover` is written later, its colour (`#eef4fa`) is the one shown, overriding the zebra stripe colour (`#f7f9fb`) for as long as the cursor stays there.

7.6 Action Buttons (Edit & Delete)

```
td button {  
  padding: 0.35rem 0.7rem;  
  margin-right: 0.35rem;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
  font-size: 0.85rem;  
}
```

These are the base styles that apply to **both** buttons (Edit and Delete) in the "Actions" column (see the HTML in the jobsheet-01 documentation):

- `border: none;` removes the default frame/border of an HTML button (usually a grey 3D look like an old Windows button).
- `border-radius: 4px;` slightly rounded corners.
- `cursor: pointer;` changes the mouse cursor into a "pointing hand" icon while it is over the button, a visual signal that this element can be clicked (HTML buttons usually do this by default, but this line guarantees consistency even if other styles change).
- `margin-right: 0.35rem;` a gap on the right of every button, so the "Edit" and "Delete" buttons do not stick together.

```
td button:first-of-type {  
  background-color: #f0ad4e;  
  color: #fff;  
}  
  
td button:last-of-type {  
  background-color: #d9534f;  
  color: #fff;  
}
```

These two rules distinguish the colours **by the order of the buttons**, not by the text/content of the buttons:

- `:first-of-type` the **first** button among buttons of the same kind inside the same cell → given an **orange/yellow** colour (`#f0ad4e`). Because the first button in the HTML is always "Edit", the Edit button is always orange on screen.
- `:last-of-type` the **last** button → given a **red** colour (`#d9534f`), following the common interface (UI) convention that red marks a risky/destructive action such as "Delete".

This is another example of the same pattern as the statistic cards in chapter 6: CSS "guesses" an element's role from its **position** in the HTML (first button = Edit, last button = Delete), not from a dedicated `class` such as `class="btn-edit"` or `class="btn-delete"`. This is concise, but it assumes the order of the buttons in the HTML **will never change** if one day you add a third button (for example "Details") between Edit and Delete, its colour may no longer be what you expect.

8. CSS: Styling the Form

This section beautifies the form in `books/add.html` dan `members/add.html` (see its structure in the jobsheet-01 HTML documentation).

8.1 The CSS Code

```
/* ===== Form ===== */
form p {
    margin-bottom: 1rem;
}

form label {
    display: block;
    margin-bottom: 0.35rem;
    font-weight: 600;
    color: #444;
}

form input,
form select {
    width: 100%;
    max-width: 400px;
    padding: 0.55rem 0.7rem;
    border: 1px solid #cdd4da;
    border-radius: 4px;
    font-size: 1rem;
}

form button[type="submit"] {
    background-color: #1d5b8a;
    color: #fff;
    border: none;
    padding: 0.6rem 1.5rem;
    border-radius: 4px;
    font-size: 1rem;
    cursor: pointer;
}

form button[type="submit"]:hover {
    background-color: #164869;
}
```

8.2 Spacing Between Fields

```
form p {
    margin-bottom: 1rem;
}
```

Remember from the HTML documentation that every form field is wrapped in a `<p>` tag. This rule puts a gap of `1rem` below **every** `<p>` inside the form, so each field (Title, Author, and so on) is clearly separated from the next one instead of being cramped together.

8.3 The Label as Its Own Block

```
form label {
    display: block;
    margin-bottom: 0.35rem;
    font-weight: 600;
    color: #444;
}
```

- `display: block;` by default, `<label>` is an *inline* element (it sits in line with the surrounding text and does not move to a new line automatically). Changing it to `block` forces the label to **always fill one whole line**, making sure the input below it (remember the HTML uses `<br>` after the label — see HTML documentation §4.3) always sits on a new line.
- `margin-bottom: 0.35rem;` a small gap between the label text and the input box below it.
- `font-weight: 600;` the label is made slightly bold, so the field name (for example "Title") stands out more than ordinary entry text, helping the user scan the form quickly.
- `color: #444;` a dark grey, slightly lighter than the body text colour (`#2b2b2b` from chapter 3), giving some colour variation without excessive contrast.

8.4 Input Boxes & Dropdowns

```
form input,
form select {
  width: 100%;
  max-width: 400px;
  padding: 0.55rem 0.7rem;
  border: 1px solid #cdd4da;
  border-radius: 4px;
  font-size: 1rem;
}
```

- The selector `form input, form select` (separated by a comma, just like `th, td` in chapter 7) applies the same styling to **all** the `<input>` elements (including `type="text"` and `type="number"`) **and** the `<select>` elements inside the form, covering every field discussed in HTML documentation §4.4.
- `width: 100%; max-width: 400px;` this combination means: the width of the input follows the width of its wrapper box (**100%**), **but** will never exceed **400px**. On a narrow screen (phone) the input shrinks to fill the screen; on a wide screen the input will not stretch unnaturally to the full width of the section (**1000px** from chapter 5).
- `border: 1px solid #cdd4da;` a thin light grey border, replacing the browser's default border, which is usually thicker/darker.
- `border-radius: 4px;` and `padding/font-size` slightly rounded corners and comfortable space inside the box, consistent with the styling of the other components (buttons, cards) on the page.

Note: this rule does **not** include `box-sizing`, but because chapter 3 already has `* { box-sizing: border-box; }` applying to every element, the `padding` and `border` on this input are automatically "eaten" from inside the `width: 100%/max-width: 400px`, rather than adding to the total width, which is the concrete benefit of the reset at the start of the file explained in chapter 3.

8.5 The Submit Button

```
form button[type="submit"] {
  background-color: #1d5b8a;
  color: #fff;
  border: none;
  padding: 0.6rem 1.5rem;
  border-radius: 4px;
  font-size: 1rem;
  cursor: pointer;
}
```

```
form button[type="submit"]:hover {  
    background-color: #164869;  
}
```

- `button[type="submit"]` is an **attribute selector** (see core concepts §1.4): it selects the `<button>` element that has an attribute `type` with exactly the value `"submit"`. This specifically targets the "Save" button (see HTML documentation §4.5), **without** touching the Edit/Delete buttons in the table, which are of type `type="button"` (HTML documentation §3.2), a real example of why writing `type="button"` explicitly in the HTML (rather than leaving the default) matters: it becomes the "marker" CSS can use to tell the kinds of button apart.
- The button is given a solid theme blue, white text, no border, and a pointer cursor,
- `:hover` changes the colour to a **darker** blue (`#164869` compared with `#1d5b8a`) while the cursor is over it, giving the same kind of visual feedback as `a:hover` (bab 3) dan `tbody tr:hover` (chapter 7).

9. CSS: Footer

The last and simplest part of `style.css`.

9.1 The CSS Code

```
/* ===== Footer ===== */
footer {
  text-align: center;
  padding: 1.25rem;
  color: #7a8794;
  font-size: 0.9rem;
}
```

9.2 Explanation

This styles the copyright text in the `<footer>` (`@ 2026 SIMPUS-Mini – Jobsheet 1`) that appears on every page:

Property	Effect
<code>text-align: center;</code>	The text is aligned to the centre of the page rather than to the left as most text is, giving the footer the feel of a "neutral" closing line rather than a lopsided one.
<code>padding: 1.25rem;</code>	A uniform gap on all four sides, giving breathing room around the copyright text.
<code>color: #7a8794;</code>	A medium grey, deliberately made fainter/less prominent than the main text colour (<code>#2b2b2b</code> from chapter 3), because copyright information is not the main information that needs to attract the user's attention.
<code>font-size: 0.9rem;</code>	A text size slightly smaller than the normal size (<code>1rem</code>), reinforcing the sense that this is secondary/supporting text.

This pattern, reducing the size and fading the colour of less important information, is the same visual hierarchy technique discussed in chapter 6 (the statistic card labels): elements that matter less functionally are given lighter visual weight, so the user's attention stays on the main content of the page.

10. Summary & Further Exercises

10.1 Overall Summary of Jobsheet 2

Part of style.css	Main Selector	CSS Concept Learned
Reset & Body	<code>*, body, a</code>	CSS reset, <code>box-sizing: border-box</code> , style inheritance (<i>inheritance</i>), pseudo-class <code>:hover</code>
Header & Navbar	<code>header, header nav ul</code>	Flexbox (<code>display: flex</code> , <code>align-items</code> , <code>justify-content</code> , <code>flex-wrap</code> , <code>gap</code>)
Main & Section	<code>main, section</code>	<code>max-width + margin: auto</code> for centring the content, <code>box-shadow</code> , <code>border-radius</code>
Statistic Cards Grid	<code>main section:nth-of-type(2)</code>	CSS Grid (<code>display: grid</code> , <code>grid-template-columns</code> , <code>fr</code> unit), positional pseudo-class <code>:nth-of-type</code>
Table	<code>table, th, td, tbody tr:nth-child(even)</code>	<code>border-collapse</code> , <i>zebra stripes</i> with <code>:nth-child(even)</code> , button styling with <code>:first-of-type/:last-of-type</code>
Form	<code>form label, form input, form select</code>	Attribute selector <code>button[type="submit"]</code> , <code>display: block</code> , responsive width limit
Footer	<code>footer</code>	Visual hierarchy through text size & colour

10.2 Core Concepts to Remember

1. **CSS is separate from HTML** and linked through `<link rel="stylesheet">`, one CSS file can be reused across many HTML pages at once (chapters 1 and 2).
2. **box-sizing: border-box** makes the width/height of an element far easier to predict when there is `padding/border` (chapter 3).
3. **Flexbox** for one-dimensional layout (the navbar), **CSS Grid** for two-dimensional column/row layout (the statistic cards), two different tools for two different needs (chapters 4 and 6).
4. **Pseudo-classes** (`:hover`, `:nth-child`, `:nth-of-type`, `:first-of-type`, `:last-of-type`) make it possible to style based on the **state** (cursor condition) or the **position** of an element, without adding any attribute to the HTML.
5. **Specificity** decides which rule wins when two CSS rules target the same element, a more specific selector (with more "conditions") usually wins, except when the specificity is equal, in which case **the order written in the file** decides (chapter 4 §4.7, chapter 7 §7.5).

10.3 How to Try It Yourself

1. Open `index.html` in the browser and compare it with how `index.html` looked in jobsheet-01 (without CSS) — notice that the navbar is now horizontal and the statistic cards are arranged in 3 columns.
2. Open the browser *DevTools* (right-click → *Inspect/Inspect Element*, or the `F12` key), click the **Elements/Inspector** tab, then click one of the elements (for example the "Total Books" card). The panel

beside it will show which CSS rules apply to that element, the best way to **see directly** the relationship between a CSS selector and what appears on screen.

3. In DevTools, try changing one of the CSS values directly (for example change `grid-template-columns: repeat(3, 1fr)` to `repeat(2, 1fr)`) and watch the effect immediately, this change is **temporary**, only in the browser, and does not change the original file, so it is safe to experiment with.

10.4 Exercises

1. **Change the colour scheme.** Replace the value `#1d5b8a` (the theme blue) throughout the `style.css` file with another colour, for example dark green, orange, or gray, then observe how that colour appears consistently in the header, the section headings, the submit button, and the table header, because they all use the same hex value.
2. **Add a fourth column.** In the statistic card grid, add one new `<article>` in the HTML (for example "Overdue Books"), then change `repeat(3, 1fr)` to `repeat(4, 1fr)` in the CSS.
3. **Add a third button to the table.** Add a "Details" button between Edit and Delete in `books/list.html`, then observe whether its colour is what you expected (remember the note in chapter 7 §7.6 about `:first-of-type/:last-of-type` being based on position, not meaning). Try to fix it by giving it a dedicated `class` if the colour is not right.
4. **Test simple responsiveness.** Narrow the browser window gradually until it is very narrow (like the width of a phone), and observe when `flex-wrap: wrap` on the navbar starts moving the menu onto a new line.